

I2C OLED Display Integration (SSD1306)

Overview

This project focuses on the development of an Inter-Integrated Circuit (I2C) bare-metal driver to interface with a 128x64 OLED display. It translates the raw environmental data into visual graphical text via a custom font dictionary.

Objectives

- * Implement strict I2C state-machine logic (START, Address, ACK, Data, STOP).
- * Configure Open-Drain GPIO pins with internal Pull-Ups.
- * Send the complex initialization sequence required to boot the OLED's internal charge pump.
- * Create a lightweight ASCII font dictionary to render text on a purely graphical display.

Components Used

- * STM32 Nucleo Board (STM32F401RE / F446RE)
- * SSD1306 128x64 OLED Display (I2C Address: 0x3C)
- * Logic level jumper wires

Working Principle

I2C is a synchronous half-duplex protocol. The master claims the bus by pulling the SDA line LOW (START condition). To communicate with the OLED, the driver must sequentially clear hardware flags (like `SB` and `ADDR`) to prevent clock stretching. Because the OLED is purely graphical, text is drawn by pushing 5x7 pixel grid bytes matching standard ASCII characters.

System Architecture

- * **I2C Driver:** Manages communication speed (100kHz Standard Mode) on PB8 (SCL) and PB9 (SDA) mapped to AF4.
- * **Safety Mechanism:** Implements a Timeout counter inside the transmission `while` loops to prevent CPU freezes if the screen is unplugged.
- * **Graphics Library:** `ssd1306.c` translates strings into hex values using a custom `fonts.c` array.

Implementation Flow

1. Initialize PB8 and PB9 as Alternate Function Open-Drain outputs.
2. Initialize I2C1 and send the 20+ byte factory boot sequence (e.g., `0x8D`, `0x14` for charge pump).

3. Clear the display memory to `0x00` (Black).
4. Move the OLED memory cursor to a specific Page (Row) and Column.
5. Transmit formatted temperature string to be rendered.

Key Observations

- * **Hardware Freezes:** If the strict sequence of reading `SR1` and `SR2` to clear the `ADDR` flag is not followed, the hardware holds the clock line LOW forever.
- * **No Backlight:** OLEDs generate their own light. Rendering `0x00` completely turns off the pixel, mimicking black, while `0xFF` provides a solid block of light.

Results

Successful rendering of live temperature and humidity data on the OLED screen, with reliable system behavior even if the physical I2C wires are temporarily disconnected.

Key Learnings

- * Hiding complex communication sequences (Start -> Address -> Data -> Stop) inside a single, robust `I2C\_MasterSendData` API.
- * Understanding the necessity of timeouts in embedded drivers when dealing with physical external wires.

 **Source Code:** <https://github.com/adlernunez/Adler-Entri/tree/master/ARM%20F401xx/FINAL%20PROJECT/STM32F446RE>

Application Layer (main.c snippet)

```
```c
#include "stm32f401re_i2c_driver.h"
#include "ssd1306.h"

int main(void) {
 I2C_Handle_t i2c1_handle = {0};

 i2c1_handle.plI2Cx = I2C1;

 i2c1_handle.I2C_Config.I2C_SCLSpeed = I2C_SCL_SPEED_SM;
 i2c1_handle.I2C_Config.I2C_AckControl = I2C_ACK_ENABLE;
```

```
I2C_Init(&i2c1_handle);
```

```
SSD1306_Init(&i2c1_handle);
```

```
SSD1306_Fill(&i2c1_handle, 0x00); // Clear screen
```

```
char display_buffer[] = "Temp: 34 C";
```

```
while(1) {
```

```
 // Clear previous row to prevent ghosting
```

```
 SSD1306_SetCursor(&i2c1_handle, 0, 2);
```

```
 SSD1306_WriteString(&i2c1_handle, " ");
```

```
 // Write new data
```

```
 SSD1306_SetCursor(&i2c1_handle, 0, 2);
```

```
 SSD1306_WriteString(&i2c1_handle, display_buffer);
```

```
 delay_ms(2000);
```

```
}
```

```
}
```

DEMO: <https://drive.google.com/drive/folders/1eXq7sDkj5dFtnOclj5tX9Nme-Wcqtehi?usp=sharing>